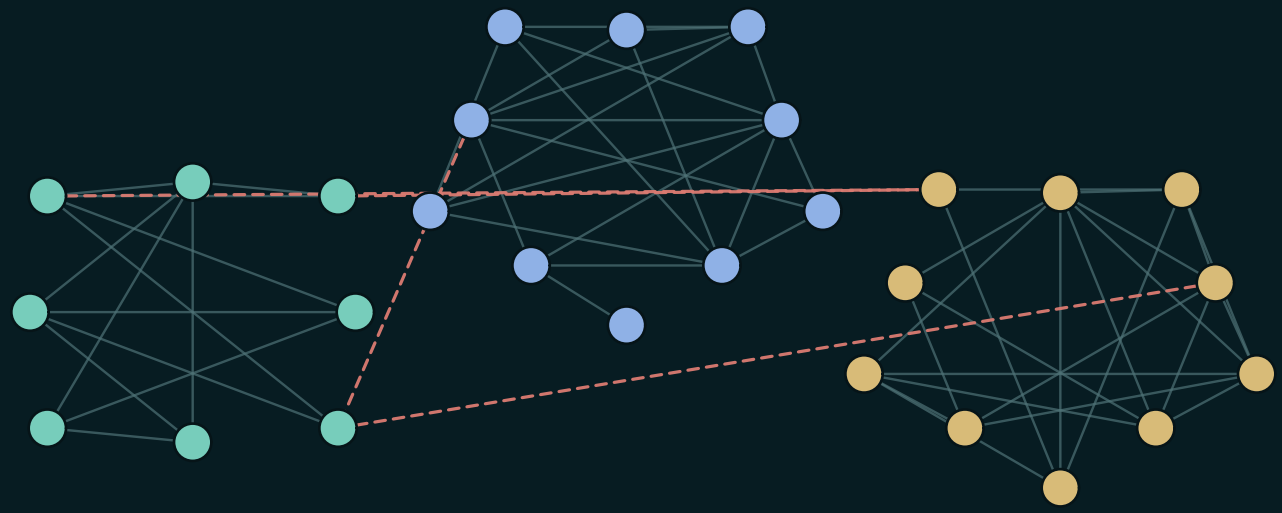
Where Do Clusters Come From?

Label propagation · a name is identity, a label is membership

Learning Clustering · original author Su3037 (Issue #4)

Circles in a contact list

- A fictional contact list: 28 people, 61 ties (the original author's data)
- Rule: see which circle your friends are in now, and join the one you are most connected to
- Repeat for a few rounds and the circles appear on their own: 8 + 10 + 10
- Raghavan, Albert & Kumara, 2007

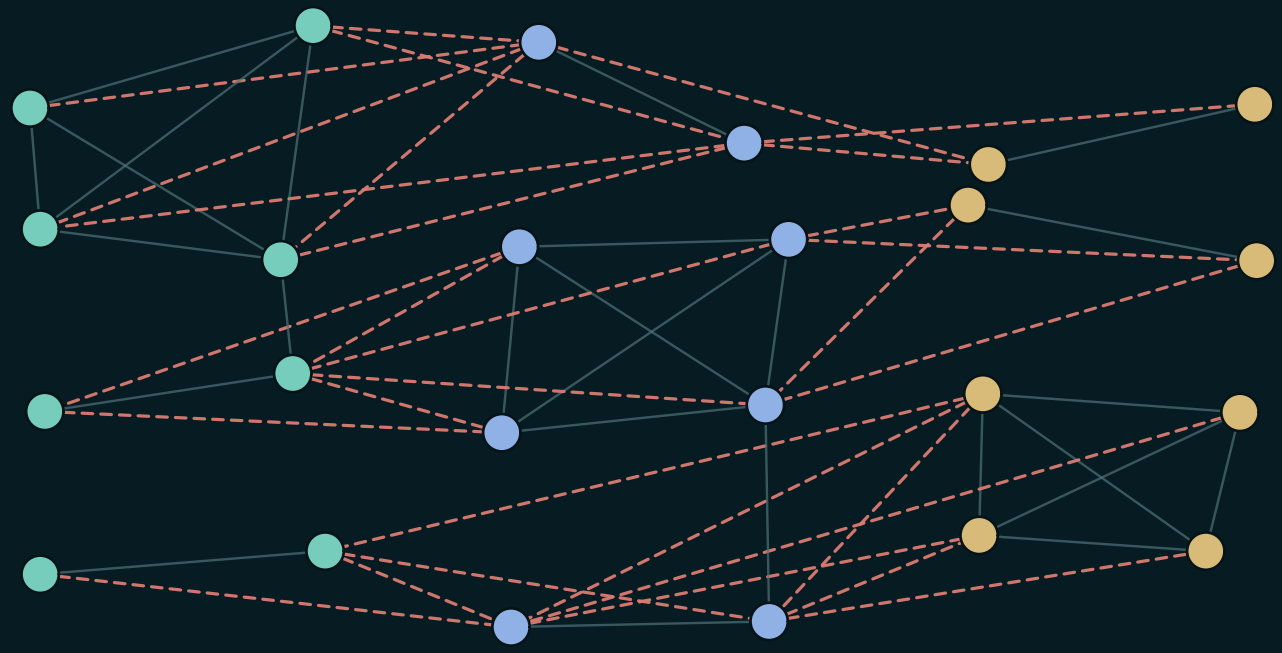


The analogy only says the structure (nodes, edges, weights) is the same; it says nothing about which circle real people belong to.

stable after 3 rounds, cut 4

The same problem in a chip

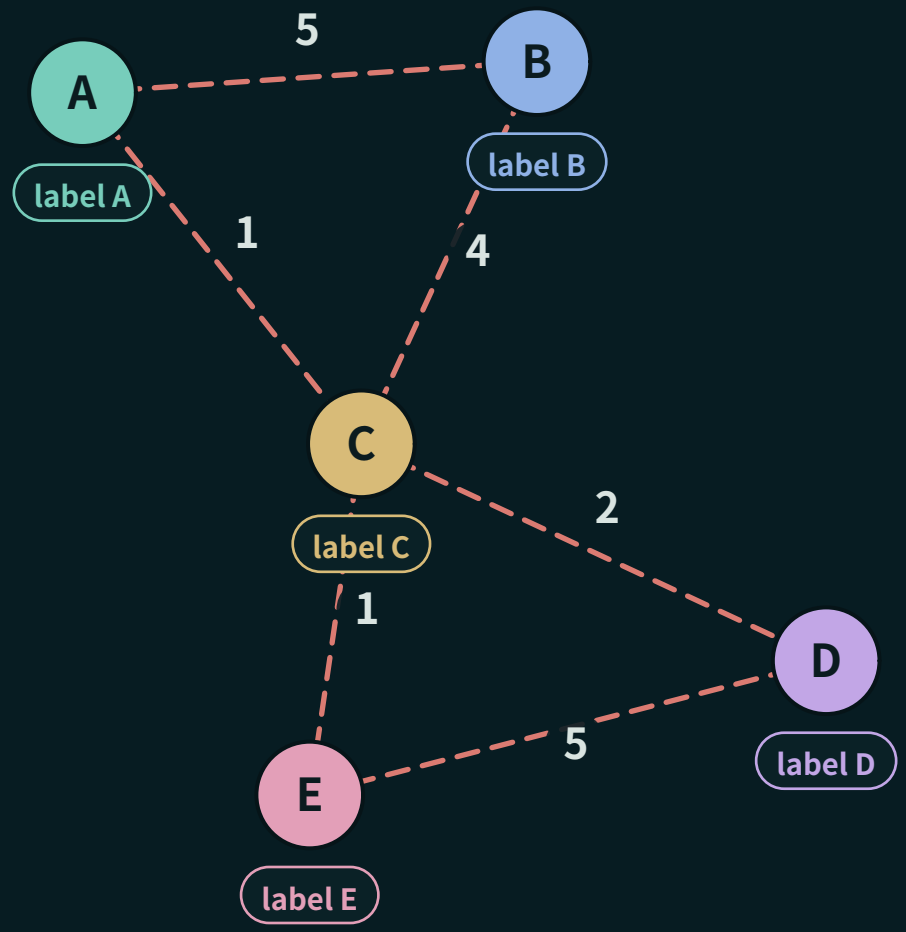
- 24 cells, 56 wires, total weight 202 (fictional)
- Cutting it into three blocks by position with a ruler cuts 88
- Brute force is hopeless: 1,577,585,295 ways to split into three equal groups
- We need a local rule that only looks at neighbours



ruler split, cut 88

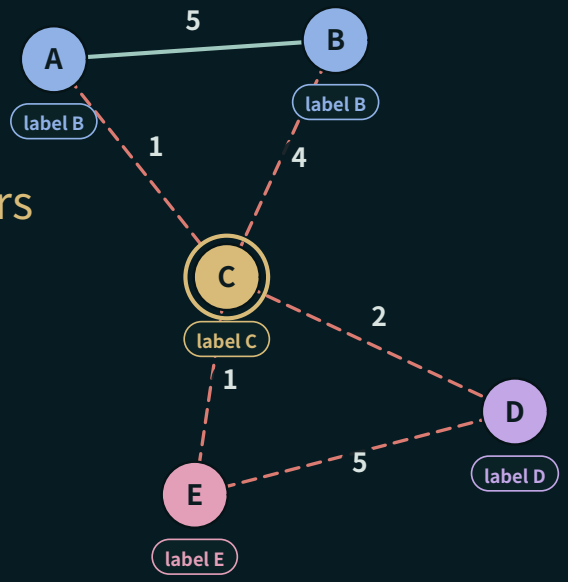
The problem

- Input: graph, visiting order, async/sync, tie rule, maximum rounds
- Start: every node's label = its own name
- Output: labels, groups, per-node votes, cut, rounds, stopping status
- No K, no merging, no balance guarantee



One update: a weighted vote

- new label(v) = the label with the largest total edge weight among v's neighbours
- C (round 1): B = 1 + 4 = 5, D = 2, E = 1
- C's label becomes B; its name is still C
- Weight counts, not the number of neighbours



C's votes (round 1)

label B	5
label D	2
label E	1

A whole round, and the quiet round

- Asynchronous: each update is written back at once
- Round 1: B B B E E, three changes
- Round 2: a full round with no change → stable
- 2 rounds in all (including the confirming round), cut 3
- Order E→A: same groups, 3 rounds

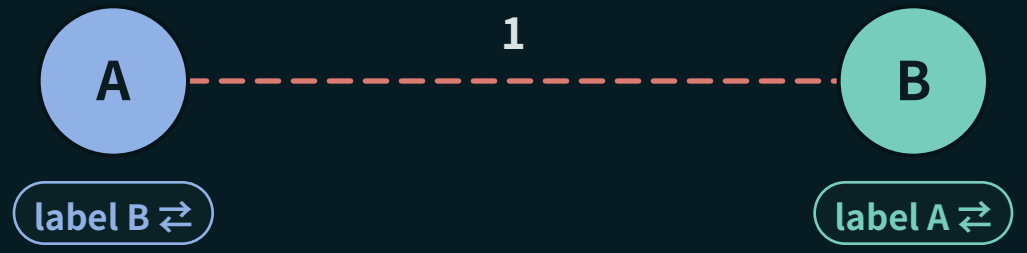
visit	votes	new label	changed
A	B:5, C:1	B	yes
B	B:5, C:4	B	no
C	B:5, D:2, E:1	B	yes
D	B:2, E:5	E	yes
E	B:1, E:5	E	no

round 2 unchanged → stable, 2 rounds, cut 3

Synchronous updates and oscillation

- Synchronous: the whole round reads one old snapshot and commits at the end
- Two nodes: A and B swap labels every round
- G01 synchronous: round 3 repeats round 1 (period 2)
- Stopping status: stable / cycle / limit
- Only stable may be called converged

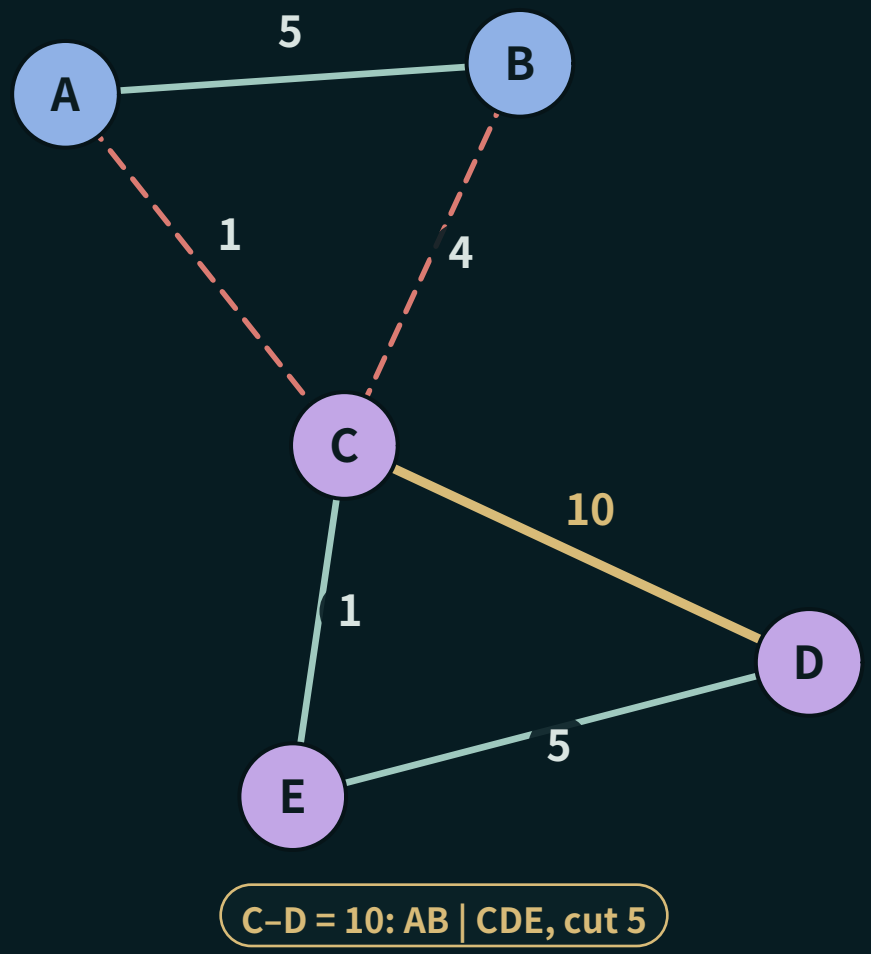
Synchronous: the two nodes swap every round



round	A	B	C	D	E
0	A	B	C	D	E
1	B	A	B	E	D
2	A	B	A	D	E
3	B	A	B	E	D

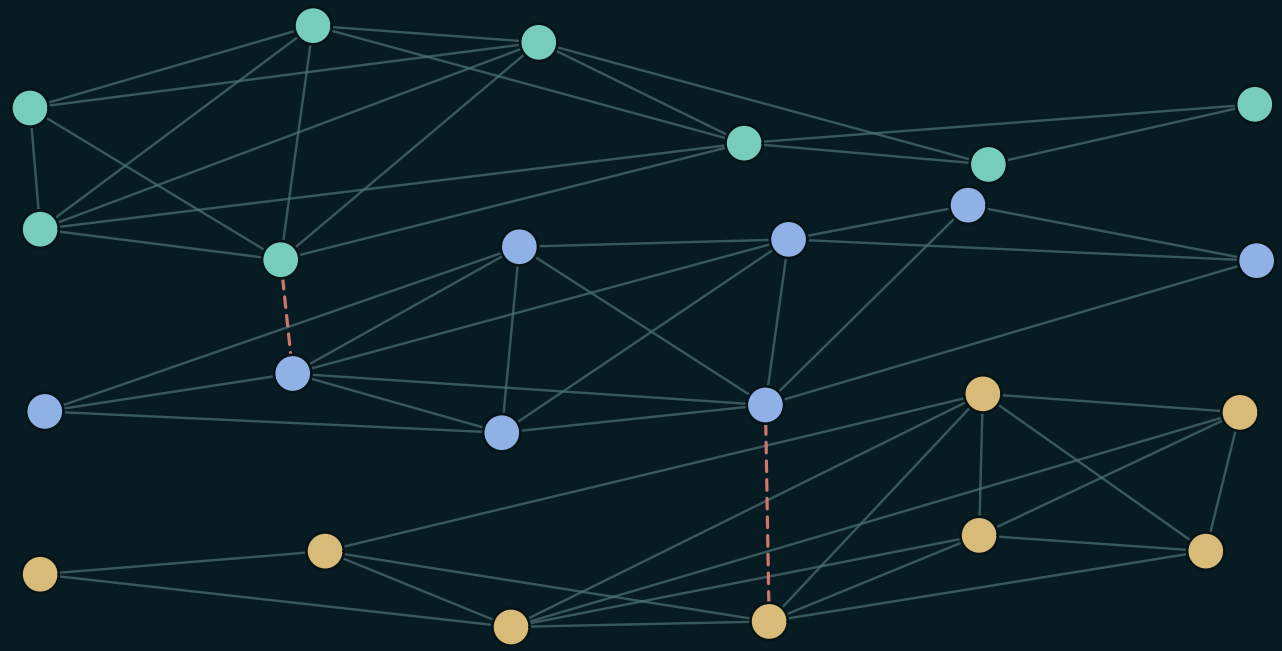
Ties and edge weights

- Tie rule: min (default) / max / keep; it must be deterministic
- Set C–D to 10 and rerun from the initial labels
- D pulls C over: AB | CDE, cut 5
- When comparing settings, always restart from the same initial state



Back to the chip

- 24-cell circuit: label propagation stops after 5 rounds
- It cuts only 2 (the ruler cuts 88)
- Colour spreads along the strongest wires
- A fictional teaching graph: says nothing about optimality in general



label propagation, 5 rounds, cut 2

The pseudocode, line by line

- Line 4: add up edge weights per neighbour label (Python: count_votes)
- Line 5: the tie rule (choose)
- Lines 6–7: the only difference between async and sync
- Line 8: no change → stable, repeat → cycle

```
1  labels[v] ← v
2  repeat each round:
3   for v in visiting order:
4     votes ← sum edge weights per neighbour label
5     new ← top vote (tie rule)
6     async: write now; sync: note it
7   sync: commit at round end
8   no change → stable; repeat → cycle
```

Common mistakes

- Treating the label as the name: "C became B"
- Counting heads: B with two votes need not beat D with weight 4
- Calling it converged after one unchanged step: it takes a whole quiet round
- Calling an oscillation "not finished" or "converged"
- Comparing results by label names: B/B/B/E/E and A/A/A/D/D are the same grouping

Summary

A name is identity, a label is membership; look at your neighbours and follow the strongest.

Only a whole quiet round is stable; synchronous updates can oscillate: report cycle.

Next module: no more local votes; order the nodes by the spectrum of the whole graph, then cut.